Blood Bank DBMS: Database and Security Report (Person A)

[redacted]

1. Database design overview

The database stores blood donors, donations, screened blood units, hospital requests and a regional stock ledger in 15 normalized PostgreSQL tables, with security enforced inside the database itself.

- **Scope:** donor registration, donation intake, laboratory screening, inventory with expiry tracking, hospital requests, first-expiry-first-out (FEFO) allocation and emergency donor alerts.
- **Entities (15):** region, role_def, component_spec, branch, hospital, app_user, donor, donor_contact, donation, blood_unit, screening_result, blood_request, request_allocation, donor_alert, stock_movement.
- **Key relationships:** a donor makes many donations; a donation yields up to one unit per component; a request is satisfied by allocating units (one unit can serve only one request); branches form a hub and local-centre tree.
- **Integrity:** primary keys, foreign keys with no cascading deletes, CHECK constraints (weight, haemoglobin, volume, coordinates, role-to-affiliation), UNIQUE constraints, and an append-only stock ledger.
- **Normalization:** all relations satisfy 3NF (13 also BCNF); the two deliberate redundancies are `blood_unit.expi[redacted]nt.r[redacted]` by the database.

The ER diagram, data dictionary and 3NF proof are attached as separate deliverables.

2. Object-relational features

PostgreSQL is an object-relational DBMS, and the schema uses its user-defined types so invalid data cannot be stored.

Feature	Where used	Benefit

Enumerated types (9)	blood_group_t, component_t, unit_status_t, request_status_t, urgency_t, test_type_t, test_result_t, alert_response_t, movement_type_t	Only valid values; compact storage; ordered comparison
Domain	phone_t (10-digit Indian mobile, starts 6-9)	One validation rule reused by every phone column
Composite type	address_t (line1, city, state, pincode)	One column holds a structured address on branch and hospital
Array type	return value of fn_compatible_donor_groups	Compatibility rules returned as a set of blood groups
JSONB	screening_result.report	Flexible lab-report documents, see section 7
User-defined functions	distance, compatibility, encryption, allocation	Logic lives next to the data

In a conventional relational design these would be lookup tables or application checks; here the database enforces them at the type level, which is the object-relational idea.

3. Discretionary access control (DAC)

Every privilege is denied by default and granted explicitly with GRANT, per table and per column, to six group roles.

Role	Clearance	Purpose
bb_admin	2	Manages users, branches, hospitals
bb_officer	2	Donations, inventory, requests, allocation
bb_lab_tech	3	Enters restricted screening results
bb_hospital	1	Places and tracks own requests
bb_donor	1	Own profile, donations, alerts
Role	Clearance	Purpose

bb_auditor	2	Read-only oversight

Design points

- **Least privilege:** writes are granted per column. A donor may update `is_available` but not `blood_group` ; a hospital may update its phone but not `is_verified` ; nobody can read `password_hash` .
- **No standing powers for the login account:** `bb_app` is created NOINHERIT and has no table access. The API acquires a role only through SET LOCAL ROLE inside a transaction, and login uses two SECURITY DEFINER functions.
- **Views as access tools:** hospitals see `v_public_inventory` (aggregate counts only), staff see `v_inventory` , and `v_request_summary` uses security_invoker so the caller's own row rules apply.
- **No hard deletes on clinical data:** DELETE is granted only to admin on branch and hospital; lifecycle is tracked with status values.
- **Execute rights:** encryption, allocation, expiry and registration functions are revoked from PUBLIC and granted only to the roles that need them.

Weakness of DAC (motivates MAC): a user holding a privilege can misuse it, so privileges alone do not stop a permitted role from seeing data above its sensitivity.

4. Mandatory access control (MAC) with Row-Level Security

Each protected row carries a classification level, each role has a clearance, and the database filters rows automatically using PostgreSQL Row-Level Security (RLS), so a forgotten WHERE clause in the application cannot leak data.

Level	Meaning	Tables
1 Internal	Operational data	blood_unit, blood_request, request_allocation
2 Confidential	Personal data	donor, donor_contact, donation, donor_alert
3 Restricted	Infectious-disease results	screening_result (forced to 3 by a CHECK)

Rule (simple security property, no read up): a row is visible and writable only if `classification <= clearance` . The clearance comes from `role_def` , looked up by `current_user` , so it cannot be forged by client-supplied values.

Own-row rules: donors see only their own donor, contact, donation, screening, alert and user rows; hospitals see only their own hospital, requests and allocations, may insert only PENDING requests under their own id, and may only cancel their own open requests.

Implementation facts

- 10 tables have RLS enabled and forced; 32 policies are defined; a role with no matching policy sees zero rows.
- Because triggers run as the table owner, each protected table has an explicit owner policy; the owner is never used by the API.
- **Demonstration:** with SELECT granted on screening_result, an officer (clearance 2) still reads 0 rows, while a lab technician (clearance 3) reads them all.
- **Practical variant:** strict Bell-LaPadula forbids writing down, which would stop officers updating level-1 inventory, so writes are allowed up to the user's own clearance.

5. Data encryption and authentication

Sensitive data is protected in three layers: hashing for passwords, field-level encryption at rest, and encrypted transport.

Layer	Technique	Detail
Passwords	bcrypt hash (one-way)	Hashed in Node; the database accepts only a valid bcrypt string and never sees a plain password
Data at rest	pgcrypto AES-256 (pgp_sym_encrypt)	Donor phone, email and address are stored as BYTEA ciphertext in donor_contact
Key handling	Session setting `app.enc_key`	Key is supplied per transaction by the API and never stored in the database; functions refuse to run without it
Randomization	Salted ciphertext	Encrypting the same text twice gives different bytes, so equal values cannot be spotted
Layer	Technique	Detail

Transport	TLS	Recommended for deployment (not enabled on local development)

Authentication and lockout: login goes through `fn_aut _auth_record_attempt` ; the fifth consecutive failure locks the account for 15 minutes, and success resets the counter. Registration is a single SECURITY DEFINER function, `fn_register_donor` , which creates the user, donor and encrypted contact atomically and normalises the email to lowercase.

Trade-off: randomized ciphertext cannot be searched with equality or indexed. The standard remedy is a blind index (a stored hash of the value) for exact-match lookups; it is noted here as future work.

Extension of the design: envelope encryption (per-record data keys wrapped by a master key) would allow key rotation without re-encrypting every row; `key_version` is alre his purpose.

6. Triggers, procedures and business rules

Business rules run inside the database as 10 triggers and stored functions, so no application bug can bypass them.

Rule	Mechanism	Error code
Donor age 18-65 (at registration and at donation)	trg_donor_age_check, trg_donation_gap_check	DONOR_NOT_ELIGIBLE
Minimum gap between donations: 90 days (male), 120 days (female), checked in both directions	trg_donation_gap_check	DONATION_TOO_SOON
Expiry date and bag code generated automatically	trg_unit_set_expiry	none
Rule	Mechanism	Error code

Status follows an allowed path only (QUARANTINE, AVAILABLE, RESERVED, ISSUED; discarded units cannot be revived)	trg_unit_status_guard	INVALID_STATUS_CHANGE
Every status change writes a ledger row	trg_unit_status_movement	none
All 5 tests negative releases units; any positive discards them	trg_screening_update_units	none
Allocation requires an available, unexpired, same-component, compatible unit and an open request	trg_allocation_validate, trg_allocation_apply	UNIT_NOT_AVAILABLE, COMPONENT_MISMATCH, INCOMPATIBLE_GROUP, REQUEST_CLOSED

Stored functions `fn_allocate_fefo` allocates the earliest-expiring compatible units, preferring

- the exact blood group to save universal donors, and uses `FOR UPDATE SKIP LOCKED` so two officers cannot take the same unit.
- `fn_compatible_donor_groups` encodes red-cell, plasma and platelet compatibility rules.
- `fn_expire_units` marks past-expiry units EXPIRED; `fn_distance_km` is a Haversine distance in pure SQL for the emergency donor search.

Concurrency: rows are locked with FOR UPDATE during validation, and a UNIQUE constraint on `request_allocation.unit_id` guarantees a unit can never be doubleallocated even under race conditions. Dates are computed in Asia/Kolkata so results are identical on every machine.

7. Semi-structured data and indexing

Laboratory reports differ by test method, so each screening result keeps a flexible JSONB document in `screening_result.report` , validated to be a JSON object by a CHECK constraint, beside the fixed columns (test type, result, tester).

- **Querying:** containment (`report @> '{"method":"ELISA"}'`) and path operators extract fields, and a GIN index with `jsonb_path_ops` makes containment queries fast.
- **XML export:** a donation or request summary can be exported as XML using PostgreSQL's `xmlelement` far ed back with XPath (`xpath()`); this is plar API layer (priority P1).
- **Why JSONB:** adding a new report field needs no schema change, while the fields that drive business logic (result, test type) remain typed columns.

Indexing (20 indexes) supports the real queries:

Index	Query served
idx_unit_fefo (partial, AVAILABLE only)	Earliest-expiry allocation
idx_unit_expiry (partial, open statuses)	Expiry sweeps
idx_donation_donor_date	Donation history, gap checks
idx_donor_group_region (partial, available donors)	Emergency donor search
idx_request_open (partial, open requests)	Pending-request queue
idx_screening_report_gin	JSONB containment
Foreign-key indexes (branch, hospital, user, allocation, alert)	Join speed

Partial indexes cover only the rows actually searched, so they are smaller and faster. A before-and-after EXPLAIN ANALYZE on `idx_unit_fefo` (run after seeding) provides the performance evidence.

8. Distributed DBMS concepts

“A second database, bloodbank_ahm, represents the Ahmedabad site. The main database registers it as a foreign server through postgres_fdw and exposes its stock table as a foreign table. The view v_network_inventory merges local and remote stock with a UNION ALL, so a single query aggregates inventory across two independent databases (location transparency, a federated design). PostgreSQL pushes the remote scan to the other site, shown as a Foreign Scan in the query plan.”

The stock ledger is horizontally fragmented by region, and the report discusses replication and the CAP trade-off as the path to a multi-node deployment.

- **Horizontal fragmentation (implemented):** `stock` artitioned by `region_code` into five physical tables (SUR, AHM, VAD, RAJ and a default partition). A query filtered by region touches only one partition (partition pruning), and the test suite confirms rows for a Surat unit land in `stock_movement_sur` .
- **Why the partition key is stored:** PostgreSQL requires the key inside the primary key, so the physical key is (movement_id, region_code).
- **Location transparency:** applications query the parent table; PostgreSQL routes rows to the right fragment.
- **Replication (discussed):** a primary with a streaming read replica would serve analytics reads while writes go to the primary. Replication is asynchronous, so a replica can briefly lag (eventual consistency).
- **Two-site demonstration (optional, planned):** `postgres_fdw` can expose a second database as foreign tables to show a distributed query across sites.

CAP position: for clinical writes such as allocation, consistency matters more than availability, because double-allocating a unit is unacceptable; the design therefore keeps all writes on a single primary (CP), accepting that a partition between sites would pause writes, while reads on replicas favour availability with possible staleness.

In a full deployment each regional partition could live on its own node, and the nearest branch's node would serve local inventory queries.

9. Testing and verification evidence

Each database layer has a rollback-only SQL test script that prints `OK` per rule and `ALL TESTS PASSED` sed on the development database.

Test file	Verifies	Result
00 setup smoke tests	8 roles (bb_app non-inheriting, none bypass RLS), pgcrypto round trip	Passed
01-02 schema	9 enums, phone domain, address type, 15 tables, 5 partitions	Passed
03 indexes	20 indexes created	Passed

20 helper functions	Haversine distance (207.7 km Surat to Ahmedabad), compatibility sets, encryption refuses to run without a key	Passed
Test file	Verifies	Result
21 triggers (about 22 checks)	Age and gap rules both directions, expiry date, status flow, ledger, screening, allocation rules	Passed
22 procedures (about 24 checks)	Registration validation and encryption, FEFO order, partial allocation, expiry sweep	Passed
50 grants	Column-level DAC, auditor read-only, no password_hash access, bb_app has no table access	all_ok = true
51 row-level security (about 33 checks)	Own-row isolation, hospital write limits, MAC demonstration	Passed

Defects found by testing: two test fixtures used 60-day-old donations that had legitimately expired (42-day shelf life); the allocation trigger correctly rejected them, which confirmed the rule works.

Performance analysis (to add after seeding): run EXPLAIN (ANALYZE, BUFFERS) on the FEFO query with and without `idx_unit_fefo` , a[illegible] time and plan type in this section.

Screenshots to attach: MAC demonstration (officer sees 0 level-3 rows), a rejected DONATION_TOO_SOON insert, and the five stock_movement partitions.

10. Limitations and conclusion

The database meets the course's security and design goals, with honest limits that a production system would address.

Limitations

- The API's login connection is the only client; anyone able to run arbitrary SQL on it could switch roles or set identity values, so every query must be parameterized and only the API may connect.
- Encrypted columns cannot be searched directly; a blind index is future work.
-

Replication and multi-node placement are discussed, not deployed; only partitioning is implemented.

- Encryption keys come from an environment variable; a production system would use a key-management service.
- Compliance wording: the design is inspired by health-data protection principles (such as India's DPDP Act 2023) and is not certified compliant. All data is synthetic.
- Medical thresholds (donation gaps, shelf lives, compatibility) follow common guidelines and should be verified against NBTC rules before citing.

Conclusion

The schema is normalized to 3NF, protected by column-level DAC and clearance-based MAC enforced inside the database, encrypts sensitive fields, enforces blood-bank business rules with triggers, allocates stock by earliest expiry safely under concurrency, and demonstrates horizontal fragmentation. Together these cover data security, objectrelational, semi-structured, distributed and advanced-SQL topics of the syllabus.